

SHADE: A Ghost-Aware Decentralized Auction Protocol (PNR2 Solution)

Sep 30, 2026 · @Harshith

1. Executive summary

SHADE (Shadow-Hedged Auction with Decaying Estimates) is a decentralized task-allocation protocol in which every robot computes every bid from signed public state, and robots it can no longer hear keep "bidding" as extrapolated ghosts.

Most decentralized auctions handle silence with a timeout: a quiet robot is dropped and its tasks are re-auctioned. That forces a bad choice. Claim fast and you may hand a task to a mediocre robot while a better one is merely out of radio range. Wait for confirmation and tasks starve. SHADE replaces the timeout with a number, called regret, and bounds it.

The five ideas

1. **Implicit bidding.** A bid is a public function of a robot's signed state manifest. No per-task bid messages are sent; any robot can compute anyone's bid at any time.
2. **Ghost bids and regret-bounded claiming.** A silent robot is replaced by an optimistic extrapolation of its bid. A robot claims a task only when the most it could be losing to an unreachable peer is below a tolerance that grows with the task's age. This gives a proven upper bound on starvation and a tunable bound on sub-optimality.
3. **Progress-ratchet leases.** A claim is a lease that is renewed only by verifiable progress, so a robot that is alive but stalled or lying loses the task without any vote.
4. **Witnessed manifests.** Physical plausibility checks, neighbour observations and equivocation proofs contain Byzantine robots without needing an honest majority.
5. **Finish-line yield rule.** When a healed partition reveals two owners, the one closer to finishing keeps the task. We prove that two honest robots can never both abandon it.

What is new, stated honestly. Market-based allocation, CRDTs, leases and signatures are all standard. To the best of my knowledge I did not find a published protocol that combines ghost bids with a regret-bounded claim rule, implicit bidding from signed manifests, and progress-ratcheted leases (see Section 16). Present it as a new synthesis, not as "the first ever", and run your own literature check before making absolute claims to judges.

Headline properties (assumptions in Section 10)

- **Bounded starvation:** an eligible task is claimed within $P + (n-1)\delta + \Gamma$ of creation, where P is a tunable patience window.
- **Bounded regret:** every claim made under partition loses at most $\epsilon(\text{age})$ bid units against the best unreachable robot.
- **No lost tasks:** two honest owners of a contested task can never both yield.
- **Bounded Byzantine damage:** each fault costs at most one lease period and earns a strike; repeat offenders are quarantined.
- **Communication cost** grows with fleet size, not with task count.

2. What the problem statement really asks

PNR2 looks like an auction problem, but its hard part is deciding what to do when you cannot hear the other bidders.

The hidden three-way trade-off. When robot R stops hearing robot P , R cannot tell whether P is dead, busy, or alive and better suited. Three quantities pull against each other:

- **Optimality:** the task goes to the best robot.
- **Latency and starvation:** the task is claimed soon and never waits forever.
- **Duplicate work:** two robots do the same task after a partition heals.

The usual answers each give one up. "Claim immediately" sacrifices optimality and creates duplicates. "Wait for a quorum" starves the smaller side of a partition, which is exactly the degraded state the PS says the fleet must survive. "Time out after T seconds" picks T arbitrarily and treats a better-but-silent robot the same as a dead one.

SHADE's stance. Choose availability during a partition, put a price on the risk (regret), bound that price with a knob, and repay it after the partition heals.

How the PS requirements map to mechanisms

PS requirement	SHADE mechanism	Section
Decentralized bidding engines, no single point of failure	Every robot runs the same bid function on signed manifests	4
Multi-variable bids: capability, battery, proximity, uncertainty, completion time	Gated weighted score with an uncertainty penalty	4

PS requirement	SHADE mechanism	Section
Robust behaviour in network partitions	Ghost bids and regret-bounded claiming	5
Recover dropped tasks without deadlock	Succession ranks, staggered takeover, lease expiry	6, 7
Prevent task starvation	Patience window: tolerance grows with task age	5
Resolve conflicting bids	G-Set ledger and finish-line yield rule	9
Byzantine hardware failures	Witnessed manifests, plausibility gate, equivocation proofs, strikes	8
Measurable, production-ready evaluation	Ablations, CBBA and central baselines, seeded fault scenarios	12, 13

Design principles

1. Decisions are pure functions of shared, signed data, so peers agree without talking.
2. Silence is information: the longer a robot is quiet, the less it can constrain us, and that is quantified.
3. Nothing is trusted because it is asserted; it is trusted because it is physically plausible and observed.
4. Every lock is a lease and every lease needs proof of progress.
5. The operator dashboard is never in the control path.

3. SHADE at a glance: one task, start to finish

Every robot runs the same agent, and a task's life is a sequence of local computations over shared data. The walkthrough uses a fleet of six robots, R1 to R6, split by a network cut so that R4 cannot hear R6.

1. **Create.** R2 senses something and issues task T7 as a signed ledger record. It gossips to all reachable peers.
2. **Compute bids.** Each live robot evaluates the public bid function on every live peer's latest manifest. No bid messages exist. All robots in R4's partition rank the same way: R4 first, R1 second.

3. **Check the ghosts.** R4 has not heard R6 for 3.1 s, so R6 is a ghost. R4 computes R6's best possible bid for T7 by assuming R6 has moved as close as its top speed allows: 0.85. R4's own bid is 0.55, so its regret is 0.30.
4. **Wait in proportion to regret.** With a bid range of $W = 1.2$ and a patience window of $P = 5$ s, R4 waits $P \times 0.30 / 1.2 = 1.25$ s. If R6 reappears in that time, exact bids decide. Otherwise R4 claims.
5. **Hold a lease.** R4 claims T7 at epoch 1. The claim is a lease that R4 renews with every manifest, and each renewal must show that its remaining cost is falling and physically possible.
6. **Fail.** R4 stalls but keeps sending manifests. Its cost stops falling, peers see the ratchet fail, and the lease lapses. R1, rank 1, waits one stagger interval and takes over at epoch 2.
7. **Heal.** The cut heals. R6 was working on T7 at epoch 1 on its side. The ledger merges and shows two owners. The finish-line rule says the one with less remaining cost keeps the task, and the other yields.

How the mechanisms fit

Stage	Mechanism	What it prevents
Compute bids	Implicit bidding (4)	Bid message storms; disagreement about scores
Wait or claim	Ghost bids, regret bound (5)	Starvation; needless sub-optimal claims
Order claims	Succession ranks (6)	Thundering herd of simultaneous claims
Hold task	Progress-ratchet lease (7)	Alive-but-idle or lying owners
Trust data	Witnessed manifests (8)	False positions, false battery, equivocation
Merge	G-Set ledger, finish-line rule (9)	Lost tasks, duplicate work, livelock

4. Mechanism 1: signed manifests and implicit bidding

A bid in SHADE is not a message. It is the output of a public function applied to a robot's latest signed manifest, so any robot can compute any peer's bid at any time.

The manifest. Each robot signs and gossips a small state record.

Field	Meaning
id, seq	Robot identity and a strictly increasing sequence number
ts	Sender's local timestamp (used only for ageing, not for ordering)
pose, velocity	Position and current velocity, plus the next few waypoints
battery E, drain rate	Energy left and its recent consumption rate
caps	Capability bitmap, for example SENSE, CARRY, MANIPULATE, RELAY
lease	Task held, epoch, remaining cost, lease expiry (Section 7)
witnesses	Signed neighbour observations of this robot (Section 8)
sig	Ed25519 signature over all fields

The bid function. For manifest M , task k and time t :

$$B(M, k, t) = G(M, k, t) \cdot \left[w_d \cdot s_{\text{dist}} + w_b \cdot s_{\text{batt}} + w_e \cdot s_{\text{eta}} + w_c \cdot s_{\text{fit}} \right] - \lambda \cdot \sigma(M, t)$$

- **G (hard gate):** 1 only if the robot has every required capability, enough battery for the trip plus a reserve, and can finish before the deadline. Otherwise the robot cannot bid.
- **s_{dist}:** 1 minus path distance to the task divided by a maximum distance, clipped to [0, 1].
- **s_{batt}:** battery left after the job and the reserve, as a fraction of capacity.
- **s_{eta}:** 1 minus (travel time plus work time) divided by the task's remaining slack.
- **s_{fit}:** the share of the robot's capabilities the task needs. This keeps specialist robots free for specialist jobs.
- **σ (uncertainty):** pose noise, link quality and manifest age. Risky robots bid less.

Default weights are 0.30, 0.25, 0.25, 0.20 and $\lambda = 0.3$. Bid values fall in a range of width $W = 1 + \lambda \sigma_{\text{max}}$.

Determinism is the whole point. Two robots must compute identical bids from identical manifests. That requires a shared map version, a fixed floating-point tolerance, and a deterministic path cost. The practical way: for each open task, every robot runs one Dijkstra pass from the task cell over the shared grid, then looks up any robot's distance in constant time. Ties break by lower robot ID.

Publishing only when needed. A robot sends a new manifest when its true state drifts from what peers would predict by dead reckoning (position error above 0.5 m or battery error above 2%), and at least once per second as a heartbeat. This is the same prediction model used for ghosts in Section 5, so live and silent robots are handled by one mechanism. The idea of publishing only on prediction error comes from dead-reckoning in distributed simulation.

Cost comparison. A conventional auction sends about N bid messages per task. SHADE sends none. Its traffic is the manifest stream, which scales with fleet size and not with task arrival rate. This is one of the metrics in Section 13.

Private vetoes. A robot may still refuse a task by publishing a manifest with reduced caps or a decline flag. Its bid then drops to zero for everyone.

5. Mechanism 2: ghost bids and regret-bounded claiming

A silent robot is replaced by the best bid it could still be making, and a live robot claims only when it could not be losing more than a tolerance that grows as the task waits. This is the core of SHADE.

Live and ghost. Robot p is live if its newest manifest is younger than τ_{live} (default 3 s, three missed heartbeats). Otherwise it is a ghost, and its manifest is frozen at the last one received.

The ghost's upper-bound bid. For a ghost last heard at time t_p , let its age be $a = t - t_p$. Build an optimistic manifest: the ghost is as close to the task as its top speed v_{max} could have carried it (distance $D - v_{\text{max}} \cdot a$, never below zero), its battery is unchanged, and its uncertainty penalty is zero. The bid function from Section 4 is then evaluated on that manifest.

$$\bar{B}_p(k, t) = B(\bar{M}_p(t), k, t), \quad \bar{D}_p = \max(0, D_p - v_{\text{max}} \cdot (t - t_p))$$

Because the bid never falls as distance shrinks or battery grows, this is an upper bound on anything the ghost could truly bid, given that its capabilities and battery cannot improve while it is away.

Which ghosts count. A ghost is a contender for task k only if it passes the gate for k , is not quarantined, and does not hold an unexpired lease (a busy robot will not take a new task). Ghosts older than T_{gone} (default 60 s) are presumed lost and dropped. Setting T_{gone} to infinity gives the strict bound below.

Regret and tolerance. Robot r 's regret on task k is how much a contending ghost could beat r 's own bid. The tolerance ε grows linearly with the age of the task until it covers the whole bid range W .

$$\rho_r(k, t) = \max\left(0, \max_{p \in \mathcal{G}_t(k)} B_p(k, t) - B_r(k, t)\right)$$

$$\varepsilon_k(t) = W \cdot \min\left(1, \frac{t - t_k^{\text{create}}}{P_k}\right), \quad P_k = \frac{\theta}{(d_k - t_k^{\text{create}})} \pi_k$$

Here d_k is the deadline, $\pi_k \geq 1$ is the task priority, and θ (default 0.25) is the patience fraction.

Claim rule. Robot r claims task k when its succession slot has arrived (Section 6) and $\rho_r(k, t) \leq \varepsilon_k(t)$.

What this buys

- **No cost in the common case.** With no ghosts, or when r already beats every ghost, regret is zero and r claims at once.
- **Waiting proportional to what could be lost.** Solving $\rho = \varepsilon$ gives a wait of $P_k \cdot \rho / W$. A regret of 0.30 with $P = 5$ s and $W = 1.2$ costs 1.25 s; a regret of 0.05 costs about 0.2 s.
- **A hard starvation bound.** Once the task is older than P_k , $\varepsilon \geq W \geq \rho$, so the best live robot may always claim.
- **A hard regret bound.** No claim is ever made with regret above ε at that moment.
- **Self-correcting.** If the ghost reappears while r waits, exact bids replace the bound and the better robot wins.

The knob. θ moves the fleet along a single trade-off curve. Section 13 turns this into a plot.

θ	Behaviour	Expected effect (hypothesis to test)
0.05	Claims almost immediately	Lowest latency, highest regret and duplicate work
0.25 (default)	Defers briefly when a stronger ghost may exist	Balanced
1.0	Waits up to a full window for silent robots	Lowest regret, higher latency under long partitions

Regret repayment. If a claim was made under positive regret and the ghost returns, the owner hands the task over when the returned robot's exact bid beats the owner's by a switching margin h (default 0.15) and the owner has done less than $\phi_{\max}$ (default 20%) of the work. The handover uses the same yield mechanism as Section 9.

Why this differs from a timeout. A timeout treats every silent robot the same and ignores the task. Ghost bids ask a specific question for each task and each silent robot: could this robot beat me here, and by how much? The answer comes from physical reachability, not from a guess about how long silence lasts.

6. Mechanism 3: succession ranks and staggered takeover

Every award produces a ranked backup list that all robots compute identically, so a failed owner is replaced by the next robot in line without a new auction round.

Ranking. When a task opens, or its lease lapses, each live robot sorts the live robots that pass the gate for that task by bid, highest first, breaking ties by lower ID. This list is the task's succession list. It needs no messages because it is a pure function of the manifests everyone already holds.

Staggered claims. The robot at rank i waits $i \cdot \delta$ before it tries to claim, and cancels if it sees a valid claim for that epoch first. Rank 0 waits nothing. The stagger δ is set to twice the recent 95th-percentile one-hop delay, with a floor of 100 ms.

$$t_{\text{claim}}(i) = t_{\text{open}} + \text{wait}_{\text{ghost}}(k) + i \cdot \delta, \quad \delta = \max(100, \text{ms}, 2 \cdot p_{95}(\text{hop delay}))$$

The ghost-regret wait from Section 5 applies to every rank, and each rank evaluates it with its own bid.

Why it helps

- **No thundering herd.** Without ranks, every capable robot claims at the same instant and the ledger fills with conflicts. With ranks, a conflict needs two robots to disagree about who is rank 0, which happens only when their views differ.
- **Fast recovery.** When an owner fails, the next rank is already known. Recovery time is the lease lapse plus $i \cdot \delta$, not lapse plus a full announce-bid-award round.
- **Aged tasks win faster.** A re-opened task keeps its original creation time, so its tolerance ϵ is already large. Recovered tasks are claimed almost immediately, which is the starvation guard for repeatedly failing tasks.

Lapse handling. A lease lapses when the verifier's own monotonic clock passes the lease expiry plus a drift margin, or when the progress checks in Section 7 fail. On lapse the claim at epoch e is marked expired locally, and the task re-enters the ranking at epoch $e + 1$.

7. Mechanism 4: progress-ratchet leases

A claim in SHADE is a lease that stays valid only while the owner shows verifiable progress, so a robot that keeps sending heartbeats but does nothing loses its task.

Why heartbeats are not enough. A plain heartbeat lease detects a dead robot. It cannot detect a robot that is alive, reports "executing", and never moves, whether from a fault or on purpose. That robot would hold the task indefinitely.

The lease record. The owner's manifest carries (task, epoch, c_{rem} , lease_until), where c_{rem} is its estimate of the remaining time to finish the task. The lease length is L (default 6 s). Every peer that receives a renewal checks two conditions before extending its local view of the lease.

1. **Ratchet.** Remaining cost must fall at least at a minimum rate over the last half-lease window. The rate is deliberately low, so obstacles and replanning do not cause false alarms.
2. **Floor.** Remaining cost cannot be below what physics allows from the owner's declared pose.

$$c_{\text{rem}}(t_2) \leq c_{\text{rem}}(t_1) - \rho_{\min} \cdot (t_2 - t_1), \quad c_{\text{rem}}(t) \geq \frac{D(\text{pose}(t), k)}{v_{\max}} + w_{\min}^{\text{rem}}(t)$$

Here $\rho_{\min}$ defaults to 0.5 (half a second of remaining cost per second) and w_{rem} is the least work time still needed, read from the task's progress counter.

Why the two checks trap a cheater. A stalled robot that reports a constant c_rem fails the ratchet. A stalled robot that reports a falling c_rem while its pose does not change eventually reports less than the floor allows. Either way, any peer that hears the manifest stream can catch it without a vote.

Work phase. After arrival, progress is the task's work counter (for example, units lifted). In simulation the task exposes a counter that nearby robots read and witness. On a real fleet, the same role is played by a site sensor or a neighbour's observation.

Consequences

- A failed ratchet or floor check does not delete the claim by itself. It stops renewals from being accepted, so the lease expires on schedule and the succession list in Section 6 takes over.
- The failing owner also earns a strike (Section 8).
- The worst delay a stalled or lying owner can impose is one lease period L plus the stagger, then one strike.

Clock safety. Leases use each robot's local monotonic clock, never wall-clock agreement. The owner treats its lease as ending $L \cdot (1 - d)$ after it last renewed, and verifiers treat it as ending $L \cdot (1 + d)$ after they last accepted a renewal, where d is the assumed maximum clock drift (default 1%). The gap between these two views means an owner stops working before a verifier lets anyone else start, except in the partition case handled by Section 9.

8. Mechanism 5: witnessed manifests and Byzantine containment

SHADE does not try to out-vote faulty robots. It limits what a faulty robot can claim, makes most lies self-evidently false, and bounds the damage of the rest to one lease period per strike.

Threat model. A Byzantine robot holds valid keys and can send anything, but it cannot forge another robot's signature, exceed its hardware limits (v_max , maximum drain rate), or change what neighbours physically observe. No honest majority is assumed.

The plausibility gate. Every received manifest passes six checks before its bid is used.

1. **Signature and sequence.** The signature is valid and seq is higher than the last accepted from that robot.
2. **Kinematics.** Position change is at most $v_max \times \Delta t$ plus a noise allowance.
3. **Energy.** Battery never rises except while docked at a charger, and never falls faster than the maximum drain rate.

4. **Witness agreement.** If a neighbour's signed observation of this robot is recent, the declared pose must match it within sensing error.
5. **Lease honesty.** The ratchet and floor from Section 7 hold.
6. **Capability stability.** Capabilities may shrink but never grow. A robot must pass its first task of each type under observation, or that capability is revoked.

A manifest that fails is ignored for bidding and adds a strike.

Witnesses. Robots within sensing range sign short observations of each other (observer, subject, subject's seq, estimated relative position). These receipts ride inside the subject's manifests and travel in gossip. A robot cannot lie about its position to a neighbour that can see it, and a false position becomes visible as soon as any robot passes nearby.

Equivocation proofs. If a robot signs two different manifests with the same seq, the pair is self-verifying evidence. Any robot that sees both gossips the pair, and every robot bans that key. Honest robots cannot be framed because a proof needs two signatures from the accused.

Strikes and quarantine. Each violation adds a strike weighted by severity. Strikes decay by one per minute. At three active strikes the robot is quarantined for five minutes, doubling on repeat. A quarantined robot's manifests are used only for monitoring: it cannot win, rank or be a contending ghost.

Attack catalogue

Attack	How it is caught	Worst effect
Position liar	Kinematics, witness disagreement	Bid ignored, one strike
Battery liar (claims more energy)	Energy monotonicity, later trip failure and lease stall	One lost lease period per strike
Capability inflation	Observed first task of each type	Task fails once, capability revoked
Bid shading within limits	Not detectable, but bounded by the gate and physical envelope	Limited to what plausible state allows
Stall after claim	Progress ratchet, floor	One lease period, one strike

Attack	How it is caught	Worst effect
Equivocation	Two signed manifests with equal seq	Key banned fleet-wide
False DONE	Completion needs a task-site receipt or a witness	Rejected; strike
Task-issue flood	Per-issuer rate limit on task creation	Excess tasks dropped

Honest limits. A lie that stays inside the physical envelope and is never observed cannot be caught by any protocol; SHADE bounds its effect but does not remove it. Section 17 lists this as a residual risk.

9. Mechanism 6: the claim ledger and the finish-line rule

The ledger is built from grow-only sets, so it merges after any partition without conflict, and duplicate owners are resolved by who is closer to finishing rather than by who has the lower ID.

The ledger. Each robot keeps a replica of four grow-only sets, all signed records merged by union.

Set	Holds
TASKS	Task definitions: id, type, location, deadline, priority, issuer
CLAIMS	(task, epoch, owner, hybrid-logical-clock time)
YIELDS	(task, epoch, owner) meaning the owner gave the task up
DONES	(task, owner, completion receipt)

Because every operation is a set union, replicas converge as soon as they hold the same records, in any order, with duplicates. Robots exchange short digests every 500 ms with two random live peers and pull whatever they are missing. When a task is done and old, its claims and yields are compacted and only a small done marker remains.

Derived state. A task is done if any valid DONES record exists. Otherwise its contenders are the robots whose latest valid manifest declares a lease on it. Using manifests rather than epochs matters: after a partition heals, an owner at epoch 1 who is 80% finished must not lose to a newcomer at epoch 2 who has just started.

The finish-line rule. When a robot sees a contender other than itself for the same task, it decides locally whether to yield. Let c_A be its own remaining cost, c_B the value in the other contender's newest record, m a hysteresis margin (default 10% of the initial cost estimate), and id the robot ID.

$$\text{A yields} \iff (c_B < c_A - m) \vee (|c_B - c_A| \leq m \wedge id_B < id_A)$$

The robot closer to finishing keeps the task. Within the margin, the lower ID keeps it. A yield is a signed record; the yielding robot stops work and returns to bidding.

Lemma 1 (at least one owner continues). Suppose honest remaining costs never rise, and that a robot which has seen the other's yield never yields itself. Then two honest contenders cannot both yield.

Proof sketch. Let A yield at time t_A on a record of B's cost from time $s \leq t_A$. Let B yield at $t_B \geq t_A$ on a record of A's cost from time u . B has not seen A's yield, so $u < t_A$, and therefore $c_A(u) \geq c_A(t_A)$ because costs never rise.

- If both yield because the other is much closer (first clause): $c_B(t_B) \leq c_B(s) < c_A(t_A) - m$, and $c_A(u) < c_B(t_B) - m$, which gives $c_A(u) < c_A(t_A) - 2m$. This contradicts $c_A(u) \geq c_A(t_A)$.
- If A yields on the first clause and B on the tie-break clause: $c_A(u) - c_B(t_B) > m$, which contradicts $|c_A(u) - c_B(t_B)| \leq m$.
- If A yields on the tie-break clause and B on the first clause: $c_A(u) \geq c_A(t_A) \geq c_B(s) - m \geq c_B(t_B) - m$, which contradicts $c_A(u) < c_B(t_B) - m$.
- If both use the tie-break clause: each would need the other to have the lower ID, which is impossible.

The case $t_B < t_A$ is symmetric. Real replanning can raise a cost slightly; choosing m above the largest replanning jump absorbs it. ■

What follows

- **No task is lost to a merge.** At least one honest owner always keeps working, and the first DONES record ends all work on the task.

- **No livelock.** Once a robot yields it never re-claims that task unless its lease later lapses through the normal path.
- **Bounded duplicate work.** Duplication requires two partitions each holding a live claim. It ends within one gossip delay and one manifest period after the heal.
- **Handover uses the same rule.** Regret repayment (Section 5) is a yield triggered by a returned robot's better exact bid.
- **A Byzantine owner that refuses to yield** either finishes the task, which is fine, or stalls and loses its lease through Section 7.

10. Formal guarantees

SHADE gives seven bounded guarantees under six explicit assumptions, and it deliberately does not promise exactly-once execution during a partition.

Assumptions

ID	Assumption
A1	Each robot has a monotonic local clock with drift at most d (default 1%). No synchronized time is needed.
A2	Public keys are distributed before the mission and signatures cannot be forged.
A3	Top speed $v_{\max}$ and maximum drain rate are known and enforced by hardware.
A4	Links are fair-lossy: a message resent often enough is eventually delivered, and gossip reaches every robot in a connected component.
A5	Task completion is observable, through a site receipt or at least one witness.
A6	For the starvation bound: at least one live, honest robot in the partition passes the gate for the task.

Guarantees

ID	Guarantee	Needs	Why it holds
G1	Convergence. Honest replicas holding the same records derive the same task state.	A2, A4	All ledger operations are set unions; derived state is a pure function of the sets.
G2	No task lost in a merge. Two honest contenders never both yield.	none beyond honest progress	Lemma 1 in Section 9.
G3	Bounded starvation. An eligible task is claimed within $P_k + (n_{\text{live}} - 1) \cdot \delta + \Gamma$ of creation, where Γ bounds gossip delay inside the partition.	A4, A6	After P_k the tolerance ε equals W , which is at least any regret, so rank 0 may claim; each lower rank adds one stagger.
G4	Bounded regret. A claim made at time t has regret at most $\varepsilon_k(t)$ against every contending ghost.	A3	The ghost bid is an upper bound, and the claim rule checks $p \leq \varepsilon$ before claiming.
G5	Bounded Byzantine delay. Each stall or lie delays a task by at most $L + (n_{\text{live}} - 1) \cdot \delta$ and earns a strike; three strikes quarantine the robot.	A2, A3	Section 7 lease expiry, Section 8 strikes. A faulty robot cannot forge another robot's records.
G6	Bounded duplicate work. After a heal, duplicate ownership ends within Γ plus one manifest period.	A4	Finish-line rule applies as soon as both contenders' records are exchanged.
G7	No deadlock. No robot ever waits on another robot's reply.	none	Every wait is a timer: patience P , stagger δ , lease L .

What SHADE does not promise

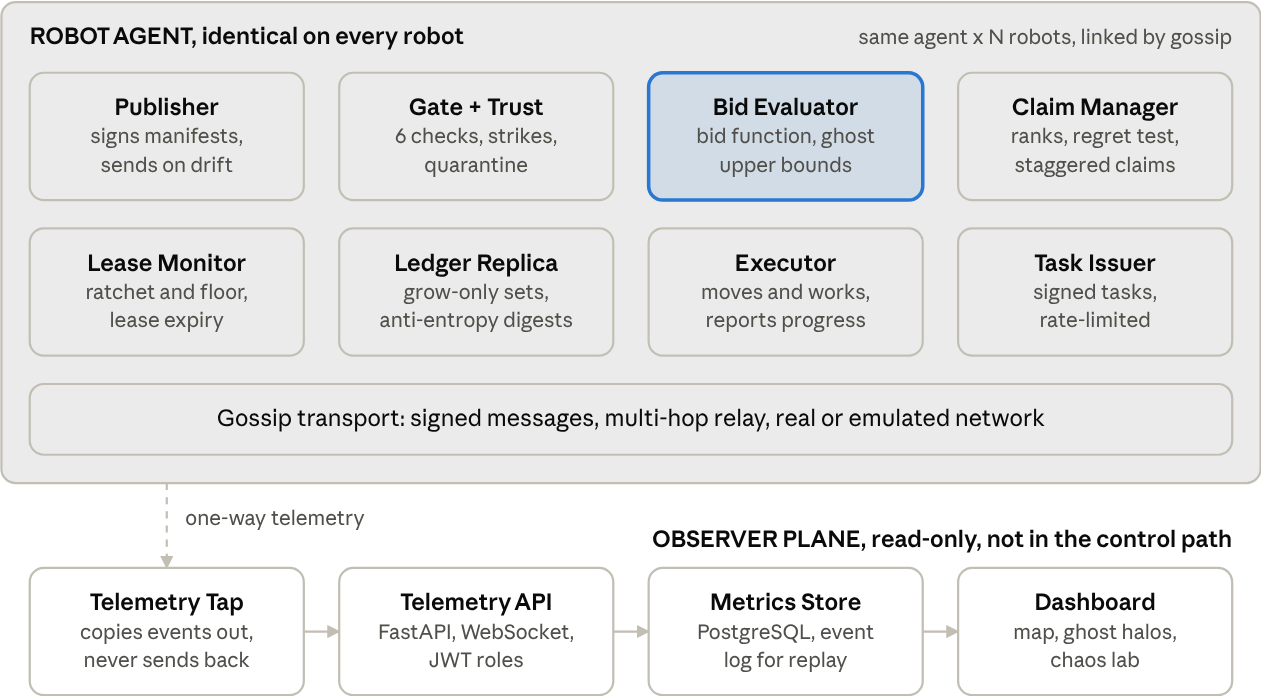
- **Exactly-once execution during a partition.** Under the availability-first choice, two partitions may each work on the same task. Tasks must be idempotent or have a cheap abort. This is the price of not starving a minority partition.
- **Global optimality.** Regret is bounded against contenders SHADE knows about, not against an unknowable optimum.

- **Protection from undetectable lies.** A lie inside the physical envelope that nobody observes is bounded, not prevented (Section 8).

11. System architecture, data model and interfaces

Each robot is a self-contained agent of eight modules, and an optional observer plane listens through a one-way telemetry tap without ever sitting in the control path.

Every robot runs the same eight modules; the dashboard only listens



Highlighted box: the ghost-bid logic, the core of SHADE.

architecture · 8 agent modules, 4 observer components

Everything in the shaded box runs on every robot. Only the telemetry tap crosses to the observer plane and it never sends anything back; operators can still issue signed tasks, which enter as ordinary ledger records.

Agent modules

Module	Responsibility	Section
Publisher	Builds and signs manifests; sends on prediction drift and at the 1 Hz floor	4

Module	Responsibility	Section
Gate + Trust	Runs the six plausibility checks, keeps strikes, applies quarantine	8
Bid Evaluator	Bid function, per-task distance fields, ghost upper bounds	4, 5
Claim Manager	Succession rank, staggered timer, regret test, claim and yield records	5, 6, 9
Lease Monitor	Ratchet and floor checks on every owner, lease expiry	7
Ledger Replica	Four grow-only sets, digest exchange, compaction	9
Executor	Moves and works; reports progress and remaining cost	7
Task Issuer	Creates signed tasks for things the robot senses, with a rate limit	9

Message types

Message	Content	Rate
MANIFEST	Signed state record (Section 4)	On drift; at least 1 Hz
LEDGER_DIGEST	Per-task epoch and record hash	Every 500 ms to 2 random live peers
LEDGER_PUSH	Missing tasks, claims, yields, done	On digest mismatch
EQUIVOCATION_PROOF	Two signed manifests with the same seq	On detection, relayed once
TASK_ISSUE	A TASKS record signed by an authorised issuer	On demand, rate-limited

Data model (observer plane, PostgreSQL)

Table	Key columns
runs	id, scenario, seed, started_at, params
robots	run_id, robot_id, type, caps, key_fingerprint
manifests	run_id, robot_id, seq, ts, pose, battery, lease, sig
tasks	run_id, task_id, type, x, y, deadline, priority, issuer
claims, yields, dones	run_id, task_id, epoch, robot_id, time
strikes	run_id, robot_id, reason, time, expires_at
events	run_id, t, kind, payload (the replayable log)
metrics	run_id, name, value, window

Interfaces (FastAPI, JWT roles viewer and operator)

Endpoint	Purpose
POST /runs	Start a scenario with a seed
GET /runs/{id}, /metrics, /events?since=	Read status, metrics and the event log
POST /runs/{id}/tasks	Issue an operator task (signed with the operator issuer key)
POST /runs/{id}/faults	Inject a fault: cut, crash, liar, stall
WS /ws/runs/{id}	Live state stream for the map

Security

- Ed25519 key per robot, generated at provisioning, listed in a registry signed by the mission authority.
- Strictly increasing sequence numbers block replay; equivocation proofs ban keys fleet-wide.
- Dashboard uses TLS and JWT with viewer and operator roles; the operator issuer key signs operator tasks.
- Task creation is rate-limited per issuer; every strike and quarantine is written to the event log.

12. Simulator, network emulator and fault injection

The simulator is a seeded, deterministic Python world in which every robot runs the real agent code, and a network emulator sits on every link, so a scenario can be replayed exactly.

World

- A 100 m × 100 m arena with rectangular obstacles on a 1 m grid. All robots share one map version and one deterministic path cost (Section 4).
- Fixed time step of 50 ms, with the agent loop running at 10 Hz inside each robot.
- Four capability flags: SENSE, CARRY, MANIPULATE, RELAY. Robot types mix them: scouts are fast and sense-only, haulers are slow and carry, generalists have several.
- Battery model: idle drain plus a movement term that grows with speed and payload. Charger stations restore energy.
- Tasks arrive by a Poisson process with a type, a location, a work quantity, a deadline and a priority. Some tasks are issued by robots that sense something; the rest come from an operator.

Network emulator. Every message passes through a link model, so the agent code never knows whether it is simulated.

- Radio range as a disk, optionally blocked by obstacles.
- Per-link random loss, base latency and jitter, plus optional reordering and duplication.
- Scripted events: partition into groups for an interval, full blackout of one robot, gradual degradation (loss ramping from 0% to 80%).

Fault injector. Faults are scheduled in the scenario file and applied inside the robot's outbound path, so honest and faulty robots use identical code apart from the injected behaviour.

Fault	Behaviour
Crash	Robot stops sending and moving
Position liar	Reports a pose closer to tasks than reality
Battery liar	Reports more energy than it has
Stall after claim	Claims, keeps sending manifests, does not move
Equivocator	Signs two manifests for one sequence number

Fault	Behaviour
Capability inflator	Claims a capability it lacks
False DONE	Announces completion without doing the work
Task flooder	Issues tasks faster than the rate limit

Scenario catalogue

ID	Scenario	Main variable
S1	Healthy baseline	None; measures cost of the protocol itself
S2	Lossy network	Packet loss from 0% to 80%
S3	Hard partition	2 or 3 groups for 30 to 120 s, then heal
S4	Flapping partition	Links cut and healed every 5 to 15 s
S5	Mid-mission crashes	10% to 30% of robots crash
S6	Byzantine mix	1 or 2 faulty robots from the fault table
S7	Scale	10, 30, 60, 100 robots at fixed task density
S8	Storm	S3 + S5 + S6 together with 50% loss

Determinism. Each run writes the seed, the scenario file, and an event log. Replaying the log reproduces the run exactly, so a judge can watch any result again.

13. Evaluation plan

The evaluation answers one question for each mechanism: does removing it make things measurably worse? Every result comes from seeded runs, so it can be reproduced.

Metrics

Metric	Definition
Completion rate	Share of tasks finished before their deadline

Metric	Definition
Mean regret	Average gap between each awarded bid and the best feasible bid at award time, computed by an oracle with full state
Time to claim	Task creation to first valid claim (p50, p95)
Starvation	Tasks unclaimed after 2× their patience window
Duplicate work	Robot-seconds spent on a task that another robot finished
Energy per task	Total fleet energy divided by tasks completed
Idle share	Fraction of robot time with no task while tasks were open
Heal convergence	Time from a heal to zero contested tasks
Messages per task	Total messages divided by tasks completed
Byzantine cost	Task-seconds lost to faulty robots before quarantine

Baselines

Baseline	What it represents
Central oracle	Hungarian assignment with perfect state and no faults; the upper reference
Central real	The same allocator behind the emulated network; shows the single point of failure
Greedy nearest	Each robot takes the closest open task it can do
Plain auction	Announce, bid, award messages with heartbeat leases and fixed timeouts
CBBA	Consensus-Based Bundle Algorithm, the standard decentralized method (Choi, Brunet, How, 2009)

Ablations of SHADE. Each one disables a single mechanism, which is what proves it is needed.

Variant	Change
No ghosts	ϵ is infinite, so robots always claim immediately
Strict ghosts	ϵ is zero and T_{gone} is infinite, so robots wait for every silent peer
No ratchet	Heartbeat-only leases
No witnesses	Plausibility checks 1 to 3 only
Lowest-ID yield	Replace the finish-line rule with a plain ID tie-break
No ranks	All capable robots claim at once

Experiments

ID	Question	Sweep	Headline plot
E1	What does SHADE cost when nothing is wrong?	S1	Completion, energy, messages vs baselines
E2	How does it degrade with packet loss?	S2, loss 0–80%	Completion rate vs loss, all methods
E3	What is the price of patience?	S3, θ from 0.05 to 1.0	Regret vs time-to-claim (a Pareto curve)
E4	Which mechanism matters most?	S8, all ablations	Bar chart of completion and duplicate work
E5	How fast does it recover from a heal?	S3, S4	Heal convergence time
E6	How much can faulty robots hurt?	S6	Byzantine cost with and without each defence
E7	Does it scale?	S7, 10–100 robots	Messages per task and time to claim vs fleet size

Hypotheses to test, not results.

- E3 produces a smooth trade-off curve in which the default θ sits near the knee.

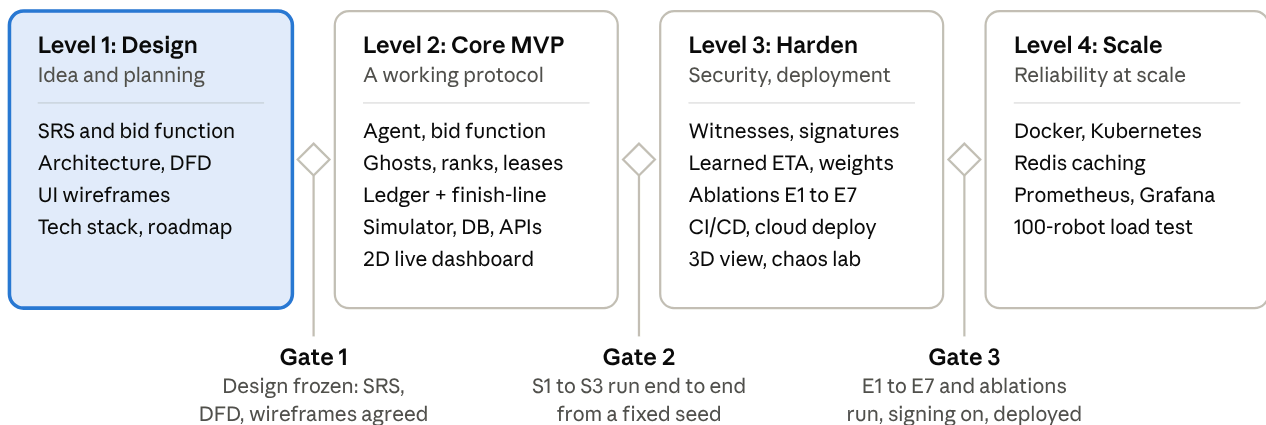
- The messages-per-task metric grows more slowly with task rate for SHADE than for the plain auction (Section 4).
- Removing the ratchet lets a stall attacker hold tasks far longer than L.
- The lowest-ID yield rule wastes more robot-seconds than the finish-line rule after long partitions.
- CBBA does well when connectivity is good and slows or stalls when the network is cut.

Statistical protocol. Run 30 seeds per configuration. Report the mean and a 95% confidence interval. Fix seeds across methods so each method sees the same tasks and faults. Publish the scenario files with the results.

14. Implementation roadmap for Levels 1 to 4

The build follows the four levels in the problem statement, and each level ends in something you can demonstrate before starting the next.

Four levels, each ending in a demonstrable gate



Highlighted band: the current level. Diamonds are the checks before moving on.

roadmap · 4 levels, 3 gates

Each gate is a pass or fail check, so a level is not started until the previous one can be shown.

Level 1: idea, architecture, planning. This document, the slide deck, the architecture and data flow diagrams, the wireframes and the tech stack.

Level 2: core functionality. Build in this order, so every step can be tested alone.

1. The deterministic bid function, the shared map and the distance fields, with a test that two agents compute identical bids.
2. The simulator, network emulator and event log.
3. Manifests, gossip transport and the ledger replica.
4. The claim manager with ranks and the ghost regret test.
5. Progress-ratchet leases and expiry.
6. The finish-line rule and yields.
7. The database schema, the FastAPI endpoints and a 2D dashboard with the task queue and chaos controls.

Level 3: intelligence, security, real deployment.

- **Advanced algorithm:** replace the static travel-time estimate with a small model trained on logged runs, and tune the bid weights offline from the same logs. The model version is part of the shared configuration and is stamped in every manifest, so all robots score with the same function; a manifest with a different version is ignored.
- **Security:** Ed25519 signing, the witness checks, equivocation proofs, strikes, JWT roles and TLS.
- **Evaluation:** the full ablation suite and experiments E1 to E7 run as a regression job on every push (GitHub Actions).
- **Deployment:** containerised API and dashboard on a cloud host, with the 3D view and ghost halos.

Level 4: scalability, reliability, enterprise readiness.

- Package simulator workers as containers and run seeds in parallel on Kubernetes.
- Cache dashboard state and metric queries in Redis.
- Export Prometheus metrics (completion rate, strikes, contested tasks) with Grafana dashboards and alerts.
- Run the 100-robot scale experiment (E7) as the load test.

15. Demo script and how to present the novelty

The demo should show one idea per minute and finish with a moment the audience remembers: a silent robot's ghost halo shrinking as it comes back into range.

Visual signature. In the dashboard, draw an expanding translucent disc around every ghost robot. The radius is $v_{\max} \times \text{age}$, the same reachable set the protocol uses for regret. Audiences see the bound, not just hear about it.

Seven-minute script

Minute	Action	What the audience sees	Message
0-1	Healthy run beside the central baseline	Both fleets complete tasks at a similar rate	The protocol costs little when nothing is wrong
1-2	Kill the central allocator	Central fleet freezes; SHADE fleet continues	No single point of failure
2-3	Cut the network into two groups	Two colour regions; ghost halos appear and grow	Silence is quantified
3-4	Create a task near a ghost	Robot waits about a second, then claims	Waiting is proportional to regret
4-5	Inject a stall attacker	Its lease stops renewing; next rank takes over	Heartbeats are not enough
5-6	Heal the network	Two owners of one task; the closer one keeps it	Finish-line rule
6-7	Show the θ trade-off plot and ablation bars	Pareto curve; each mechanism's value	Every part is measured

Three sentences for judges

1. "Robots don't send bids; they compute everyone's bid from signed state, so silence has a price we can calculate."
2. "When a peer goes quiet, we assume the best it could be doing and only claim if we can't lose more than a tolerance that grows with the task's age."
3. "A task is a lease that needs proof of progress, so a robot that is alive but stuck loses it automatically."

Likely questions

Question	Short answer
Isn't this just CBBA?	CBBA reaches agreement by exchanging winning bids. SHADE avoids bid messages and adds a computed bound on what silence can cost. We compare against CBBA in E2 and E7.
What about duplicate work?	It can happen during a partition, is bounded, and ends by the finish-line rule after heal (G6).
Can a malicious robot break it?	It can delay a task by one lease per strike and is then quarantined (G5). Lies inside the physical envelope are bounded, not eliminated.
Why not use a blockchain or Raft?	Both require a quorum, which a small partition cannot reach. SHADE stays available on every side.
Is it proven?	The lemma and bounds are proven under stated assumptions (Section 10). Performance claims come from the seeded experiments.

Backup plan. Record one clean run of every scenario. If the live demo fails, play the recording and replay the event log to show it is reproducible.

16. Related work and honest novelty assessment

The closest published ideas either predict teammates' bids or make claims more cautious under bad links; none I found bounds the cost of silence with a regret tolerance tied to task age. My search was short (a handful of queries), so treat this as a starting list, not a proof of absence.

Closest neighbours

Work	What it does	How SHADE differs
FORMICA, Lopez, Muirhead, Pincioli	Learns to predict teammates' bids so robots coordinate with no communication, training against task-allocation regret	SHADE uses an analytic, optimistic bound from physical reachability, applies it as a guard on claims, and keeps using real messages when they arrive

Work	What it does	How SHADE differs
TGPR, a preprint on trust-gated predictive reallocation	Estimates per-robot link reliability, discounts unreachable bidders, and lets idle robots make confidence-gated claims after a stalled award	SHADE has no coordinator to stall, derives its wait from regret and task age, and proves a starvation bound. The preprint itself reports lower completion under poor channels, which SHADE's bound is designed to avoid
Consensus-Based Bundle Algorithm	Decentralized auction; robots exchange winning bids and reach conflict-free assignment with a worst-case guarantee	SHADE sends no bid messages, and does not need agreement before acting. It handles partitions by design rather than by waiting for consensus
Repeated auctions, Nanjanath and Gini	Re-auctions remaining tasks after each completion to recover from failures	Recovery here is continuous and precomputed through succession ranks
Sariel, Balch, Erdogan	Auction-based framework robust to message loss and robot failure, with dynamic re-allocation and recovery routines	No progress-verified leases, no Byzantine model, no bound on starvation
MURDOCH	Fitness-based auctions over publish/subscribe	Classic baseline family for the plain auction
Prim Allocation	Simple auction with a $2\times$ cost guarantee	Optimises cost, not behaviour under silence
Mirzaei and Esposito	Shows consensus-auction protocols can be pushed into oscillation by malicious rebidding	Motivates the plausibility gate and strikes

Standard techniques SHADE builds on (from general knowledge, not looked up here): grow-only set CRDTs, leases, hybrid logical clocks, Ed25519 signatures, equivocation proofs, and dead-reckoning update suppression from distributed simulation.

Honest assessment

- **Likely new as a combination:** a ghost upper-bound bid used to bound the regret of a claim, with a tolerance that grows with task age so that starvation is bounded; progress-ratchet leases with a physical floor; and the finish-line yield rule with a no-mutual-yield lemma.
- **Not new individually:** predicting others' bids, regret as a metric, leases, CRDT ledgers, plausibility checking.
- **What to say:** "a new synthesis with proven bounds", never "the first". Before you present, run your own searches on terms such as ghost bid, regret-bounded claim, lease with progress, and decentralized task allocation under partition, and add anything you find to this table.

17. Risks, limitations and mitigations

The biggest risks are scope and tuning, not the theory; the mitigations below keep a working demo possible even if the full protocol is only partly built.

Risk	Why it matters	Mitigation
Too much to build	Six mechanisms plus simulator, dashboard and experiments	Build in layers (Section 14); each layer is demonstrable alone, and ablation flags let you show partial systems honestly
Bid non-determinism	Different bids on different robots break agreement	One shared map version, deterministic Dijkstra, fixed tie-breaks, tolerance on float comparison; unit-test that two agents compute identical bids
Ghost bound too loose	An over-optimistic bound makes robots wait for peers that could not really help	Exclude busy ghosts, drop ghosts after T_{gone} , cap the wait by P_k ; measure in E3

Risk	Why it matters	Mitigation
Parameter tuning	θ , L , $\rho_{\min}$, m interact	Ship defaults from Appendix; run the θ sweep and one sensitivity sweep per parameter
Duplicate work under partition	Real robots may waste energy or collide	State the trade-off openly; require idempotent or abortable tasks; measure duplicate work in every experiment
Undetectable lies	A lie inside the physical envelope that nobody observes	Bounded by the gate; report it as a residual risk, not a solved problem
Single-task assumption	Real robots queue several tasks	Treat bundles as future work; a one-deep look-ahead queue is a small extension
Real-world sensing	Witness receipts assume reliable neighbour observation	In simulation this is exact; on hardware, add noise models and widen the tolerance
Claims of novelty	Someone may have published something close	Use the wording in Section 16 and finish your own literature check

Scope cuts if time runs out. In order: drop the 3D viewer (keep the 2D map), drop the ROS 2 bridge, drop the Kubernetes step, drop witnesses (keep checks 1 to 3 and the ratchet), and keep implicit bidding, ghost bids, ranks and the finish-line rule. Those four are the identity of the project.

18. Appendix: pseudocode and default parameters

The pseudocode below is the whole agent loop; each function maps to one mechanism, so it doubles as a module list for the codebase.

```
# Runs on every robot at 10 Hz. All decisions use only local data.
def tick(now):
    publish_manifest_if_needed(now)                # dead-reckoning suppression, 1
    Hz floor (Sec 4)
    for m in inbox.manifests():
        if gate.accept(m):                          # checks 1-6 (Sec 8)
```

```

        view.update(m)
    else:
        trust.strike(m.id, gate.reason)
    ledger.merge(inbox.records())           # G-Set union (Sec 9)
    view.classify(now)                     # live vs ghost by manifest age
    for k in ledger.open_tasks():
        claim_step(k, now)
    lease_step(now)
    contest_step(now)
    execute_step(now)

def claim_step(k, now):
    live = [p for p in view.live() if gate_ok(p.manifest, k, now)]
    ranked = sorted(live, key=lambda p: (-bid(p.manifest, k, now), p.id))
    if me.id not in [p.id for p in ranked]:
        return
    i = [p.id for p in ranked].index(me.id)
    if now < k.open_time + i * DELTA:      # staggered takeover (Sec 6)
        return
    if regret(k, now) <= eps(k, now):      # regret-bounded claim (Sec 5)
        ledger.add_claim(k, epoch=k.epoch + 1, owner=me.id)

def regret(k, now):
    ghosts = [g for g in view.ghosts() if eligible(g, k, now)]
    ub = max((ghost_bid_ub(g, k, now) for g in ghosts), default=float('-inf'))
    return max(0.0, ub - bid(me.manifest, k, now))

def ghost_bid_ub(g, k, now):
    age = now - g.manifest.ts
    d = max(0.0, dist_field[k].at(g.manifest.pose) - VMAX * age)  # closest
    it could be
    return bid_from(d, g.manifest.battery, sigma=0.0, caps=g.manifest.caps,
task=k, now=now)

def eps(k, now):
    P = THETA * (k.deadline - k.created) / k.priority
    return W * min(1.0, (now - k.created) / P)

def lease_step(now):
    for p in view.live():
        if p.lease and not (ratchet_ok(p) and floor_ok(p)):  # progress
ratchet (Sec 7)

```

```

        trust.strike(p.id, 'lease')
        view.expire_lease(p)
    if me.lease and making_progress():
        me.renew_lease(now)

def contest_step(now):
    k = me.task
    for q in contenders(k) - {me.id}:
        if ledger.has_yield(k, q):
            continue
        cA, cB = me.c_rem, view[q].lease.c_rem
        if cB < cA - M or (abs(cB - cA) <= M and q < me.id):
            ledger.add_yield(k, me.id)
            me.release(k)
            break
# finish-line rule (Sec 9)

```

Default parameters

Parameter	Default	Meaning
τ_{live}	3 s	Manifest age after which a robot becomes a ghost
Heartbeat floor	1 Hz	Minimum manifest rate
Publish thresholds	0.5 m, 2% battery	Dead-reckoning error that triggers a manifest
Bid weights (w_d, w_b, w_e, w_c)	0.30, 0.25, 0.25, 0.20	Proximity, battery, ETA, capability fit
λ	0.3	Uncertainty penalty
θ	0.25	Patience fraction of the task window
T_{gone}	60 s	Ghost dropped from contenders
δ	$\max(100 \text{ ms}, 2 \times \text{p95 hop delay})$	Stagger between ranks
L	6 s	Lease length
ρ_{min}	0.5	Minimum remaining-cost decline rate

Parameter	Default	Meaning
d	1%	Assumed clock drift
m	10% of initial cost	Finish-line hysteresis margin
h, $\phi_{\max}$	0.15, 20%	Handover bid margin and maximum progress for handover
Strikes	3 to quarantine, 5 min, doubling, decay 1/min	Trust policy
Gossip	Digest every 500 ms, fan-out 2	Anti-entropy
Task-issue limit	1 per second per issuer, burst 5	Flood protection